AI Generalist | Applied AI Builder

Candidate Practical Assignment — Submission Document

Candidate: Tushar Anil Lawande | Date: April 17, 2026 | Project: DDR AI Report Generation System

---

🏗️ What I Built — Project Overview

I built a production-grade, multi-stage AI pipeline that automatically converts raw property inspection and thermal imaging PDFs into a professionally formatted, client-ready Detailed Diagnostic Report (DDR).

This solves a real-world problem in the property inspection industry: inspectors spend hours manually writing structured reports from raw field notes and thermal camera outputs. This system automates the entire workflow in under 10 seconds.

Tech Stack

Layer	Technology
Language	Python 3.11
AI / LLM	Google Gemini 1.5 Pro (with offline fallback)
PDF Extraction	PyMuPDF (fitz)
Data Modeling	Pydantic v2
Visualization	Matplotlib
Report Rendering	Jinja2 (dark-themed HTML template)
Web Interface	Streamlit
Testing	Pytest — 35+ unit tests

🧠 How It Works — Pipeline Architecture

The system runs a 3-stage AI reasoning chain:

```
[Inspection PDF]  ──▶ PDF Extractor (PyMuPDF) ──▶ Full Text + 150 Site Photos
[Thermal PDF]     ──▶ PDF Extractor ──▶ Thermal Parser ──▶ 30 ThermalReading objects
                                                  │
                  ┌──────────────────────────────────────────┐
                  │          Chart Generator (matplotlib)    │
                  │  Thermal Profile · ΔT Trend · Severity Donut  │
                  │  Confidence Bar Chart                    │
                  └──────────────────────────────────────────┘
                                                  │
                  ┌──────────────────────────────────────────┐
```

```
│                LLM Engine v2                        │
│  Stage 1: Structured Extraction                     │
│  Stage 2: Thermal Integration + Analysis            │
│  Stage 3: Final DDR Synthesis                       │
│  Fallback: Advanced Rule-based Engine               │
└─────────────────────────────────────────────────────┘
                                    │
┌─────────────────────────────────────────────────────┐
│          Report Builder (Jinja2)                    │
│  Self-contained HTML  ·  Structured JSON            │
└─────────────────────────────────────────────────────┘
```

Stage 1 — Structured Extraction

The LLM reads the raw inspection text and extracts every observation into a structured JSON format: area name, negative-side findings, positive-side (source) findings, photo references, and metadata such as inspection date and inspector names.

Stage 2 — Analysis + Thermal Integration

The LLM receives the Stage 1 structured data alongside 30 thermal camera readings. For each area it reasons through: what the ΔT data signals about moisture, whether thermal data confirms or contradicts the visual inspection, a severity level (Critical / High / Moderate / Low) with explicit chain-of-thought, and a confidence score (0.0–1.0) per observation.

Stage 3 — Final Synthesis

The LLM synthesizes all findings into: a 2–3 sentence executive client summary, prioritized root causes with evidence, 11 graded recommended actions (Immediate / Short-term / Medium-term / Long-term), conflict detection, and missing-information flags.

Offline Fallback

If no Gemini API key is provided the system automatically activates an advanced rule-based engine that applies domain logic, pattern matching, and ΔT threshold detection — the pipeline still produces a complete, accurate report.

📊 DDR Output — 7 Sections

#	Section	Contents
1	Property Issue Summary	Executive 2–3 sentence overview for the client
2	Area-wise Observations	7 areas — neg/pos sides, site photos, confidence scores, AI reasoning trace
3	Probable Root Causes	6 evidenced, ranked root causes
4	Severity Assessment	Per-area severity table + AI confidence + donut chart
5	Recommended Actions	3 immediate + 4 short-term + 2 medium + 2 long-term actions
6	Additional Notes	Conflicts, cross-flat issues, thermal methodology

7	Missing / Unclear Info	7 explicit 'Not Available' entries

⚙️ Advanced Features

Feature	Detail
3-Stage LLM Chain	Extract → Analyze+Merge → Synthesize (Gemini 1.5 Pro)
Offline Mode	Advanced rule-based engine — no API key required
Confidence Scoring	0–1 per observation, tracked from evidence sources
Chain-of-Thought	Full AI reasoning trace embedded per section
Anomaly Detection	ΔT > 3°C = Caution; ΔT > 5°C = High-Risk
Conflict Detection	Flags inspection vs thermal data mismatches
4 Analytical Charts	All generated with matplotlib, embedded as PNG
Image Extraction	150 photos extracted, 37 placed in relevant sections
Self-contained HTML	Single ~1 MB file — zero external dependencies
Unit Tests	35+ tests across all modules (pytest)
Structured Logging	Coloured console + file logging
Generalizable	Works on any similar inspection + thermal PDF format

---

📈 Sample Results (Provided Documents)

Metric	Value
Input documents	Sample Report.pdf (23 pages, 150 images) + Thermal Images.pdf
Pipeline mode	Advanced Rule-based Engine (offline)
Processing time	~9.8 seconds
Photos extracted	150 site photos
Thermal readings	30 / 30 parsed — all 30 anomalous (avg ΔT = 5.01°C)
Areas analysed	7 (Hall, 2 Bedrooms, Kitchen, External Wall, Parking, Bathroom)
Overall severity	CRITICAL — active slab leakage in parking area
AI confidence	86%
Actions issued	11 (3 immediate · 4 short-term · 2 medium-term · 2 long-term)
Report size	1,032 KB self-contained HTML

⚠️ Limitations

- **1.** PDF Format Dependency —

The thermal parser uses regex patterns tuned to the GTC 400 C camera export format. Different manufacturers or PDF layouts require parser adjustments.

- **2.** Image Attribution —

Photo-to-area mapping is based on photo numbering sequence and LLM reasoning. Non-sequential photo labels can cause misplacements.

- **3.** LLM Hallucination Risk —

In Gemini mode the LLM could occasionally infer findings not in the source. Mitigated by strict prompting ('Do not invent / use Not Available') and rule-based fallback.

- **4.** Offline Mode Coverage —

The rule-based fallback is calibrated to the provided sample documents. Very different property types (industrial, heritage) may need re-tuning without a Gemini key.

- **5.** Single Language / Residential Only —

Currently handles English-language residential flat inspection reports. Multi-language or commercial property formats are not yet supported.

- **6.** No Incremental Editing —

The Streamlit UI requires a full pipeline re-run for any change. Section-level editing is not yet implemented.

🚀 How I Would Improve It

Short-Term (Next 2 Weeks)

- Fine-tune an extraction model on 50–100 real inspection reports for higher accuracy and consistency.
- Extend thermal parser to handle FLIR, Seek, and other major thermal camera export formats.
- Add PDF export (WeasyPrint / headless Chrome) for a professional print-ready output.

Medium-Term (1–2 Months)

- Vector database (Pinecone / ChromaDB) for RAG — compare new findings against historical DDRs.
- Human-in-the-loop review — Streamlit editor for inspectors to approve/edit each AI section.
- Multi-document trend reports — accept multiple inspection rounds for change-over-time analysis.
- Docker containerisation + cloud deployment (Google Cloud Run) for scalable on-demand processing.

Long-Term (3+ Months)

- Fine-tune Gemma 7B on proprietary inspection data — reduce API costs, improve domain accuracy.
- Auto-generated repair quotations integrated with a construction cost database.
- Mobile companion app — field inspectors capture & tag photos in real time, feeding directly into the pipeline.

🛠️ How to Run

Option A — CLI (No Setup Required)

```
cd DDR_AI_System
pip install -r requirements.txt
python generate_report.py
# Output: output/DDR_Report.html  (open in any browser)
```

Option B — Streamlit Web App

```
python -m streamlit run app.py
# Open: http://localhost:8501
```

Run Tests

```
python -m pytest tests/ -v
# Expected: 35+ tests, all passing
```